



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

Tutorial 4 : Generative AI

NAME	MATRIC NO
MUHAMMAD NAIM BIN ABDULLAH	A23CS0134
MUHAMMAD ADAM BIN RAZALI	A23CS0116
AFIF SHAQIR IRFAN BIN ARQAM	A23CS0204

1.0 Introduction

This tutorial demonstrates the integration of Generative AI (Claude) into a real-world dataengineering pipeline. The project uses the Dirty Cafe Sales dataset which is a synthetic dataset from Kaggle containing 10,000 transactions intentionally seeded with data quality issues such as missing values, invalid ERROR/UNKNOWN placeholders, inconsistent date formats, and duplicate rows. The goal is to build a complete ETL (Extract, Transform, Load) pipeline with AI assistance, where Claude generates each stage of the pipeline code from natural-language prompts. This explores how AI can accelerate data engineering tasks while still requiring human oversight for validation and correctness.

2.0 Objectives

- Design and implement a 4-stage ETL data pipeline for cafe sales data.
- Leverage Claude (Generative AI) to generate Python code for each pipeline stage.
- Identify and resolve data quality issues (missing values, duplicates, type errors).
- Load cleaned data into a SQLite database and generate business insights.
- Demonstrate prompt engineering as a practical data engineering skill.

3.0 Use case

The use case as in Figure 1 below summarises the complete pipeline from a functional perspective. Two actors interact with the system: the Data Engineer, who triggers the pipeline and validates its outputs, and Claude AI, which generates the code for every stage. The system boundary represents the full ETL pipeline (`run_pipeline.py`), enclosing five internal processes that execute in sequence.

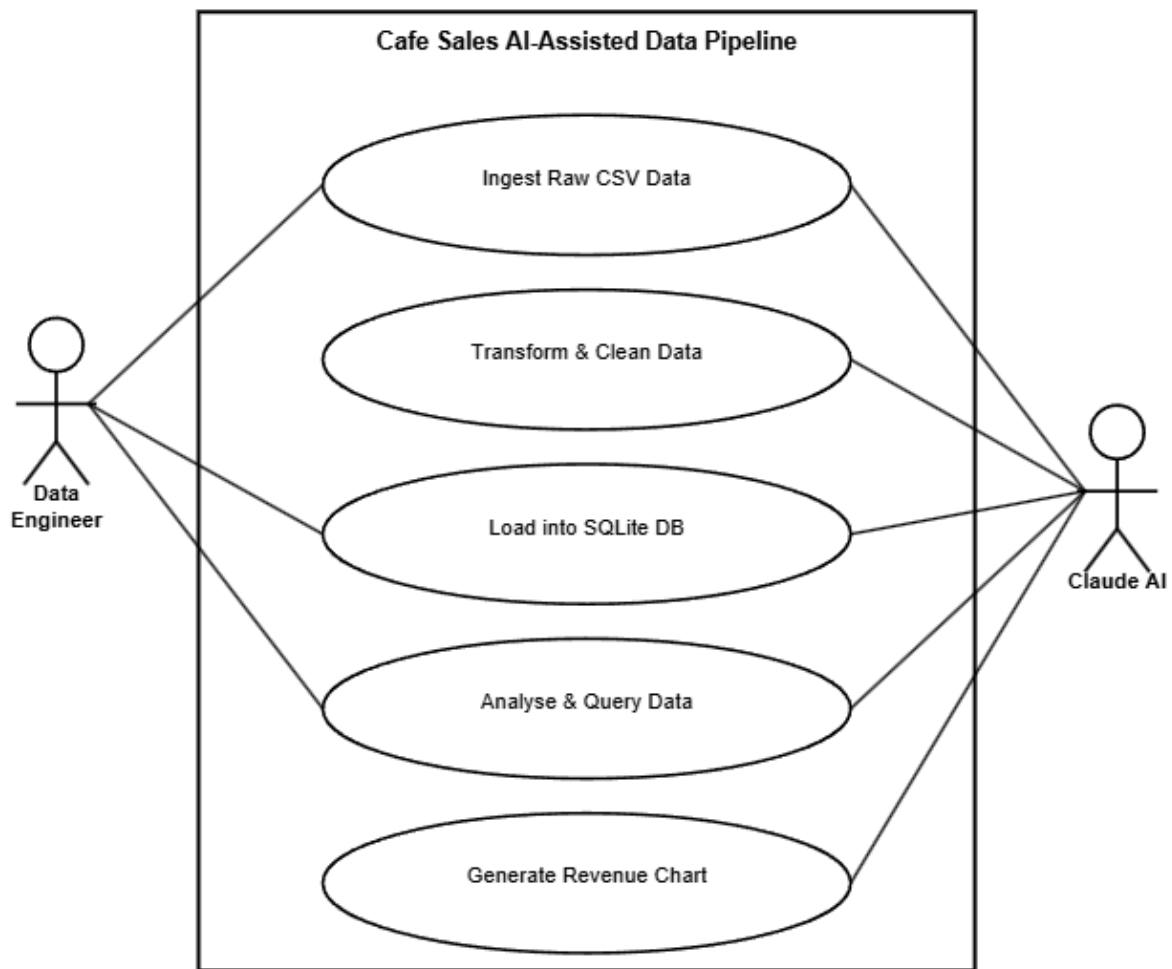


Figure 1: Use case diagram for the Cafe Sales AI-Assisted Data Pipeline

4.0 Dataset Overview

The Dirty Cafe Sales dataset contains 2,020 raw records (after deduplication: 2,000 rows) across 8 columns. Key data quality issues present in the raw file as in Table 1:

Column	Issue Type	Handling Strategy
Item	NULL?UNKNOWN placeholders	Fill with 'Unknown'
Quantity	ERROR strings,Nan	Convert to numeric;fill with median
Price Per Unit	ERROR strings,Nan	CONvert to numeric;fill with median
Total Spent	Missing/Inconsistent	Recalculate from Qty x Price;fill median
Transaction Date	Unparseable/malformed	Drop invalid rows
Payment Date	UNKNOWN placeholders	Fill with 'Unknown'
Location	UNKNOWN placeholders	Fill with 'Unknown'

Table 1: represents overview of Dirty Cafe Sales dataset

5.0 Pipeline Structure

The pipeline as is *Figure 2* below follows a classic ETL architecture broken into four sequential Python scripts, each with a single responsibility. An orchestrator script (`run_pipeline.py`) chains them together and halts execution if any stage fails.

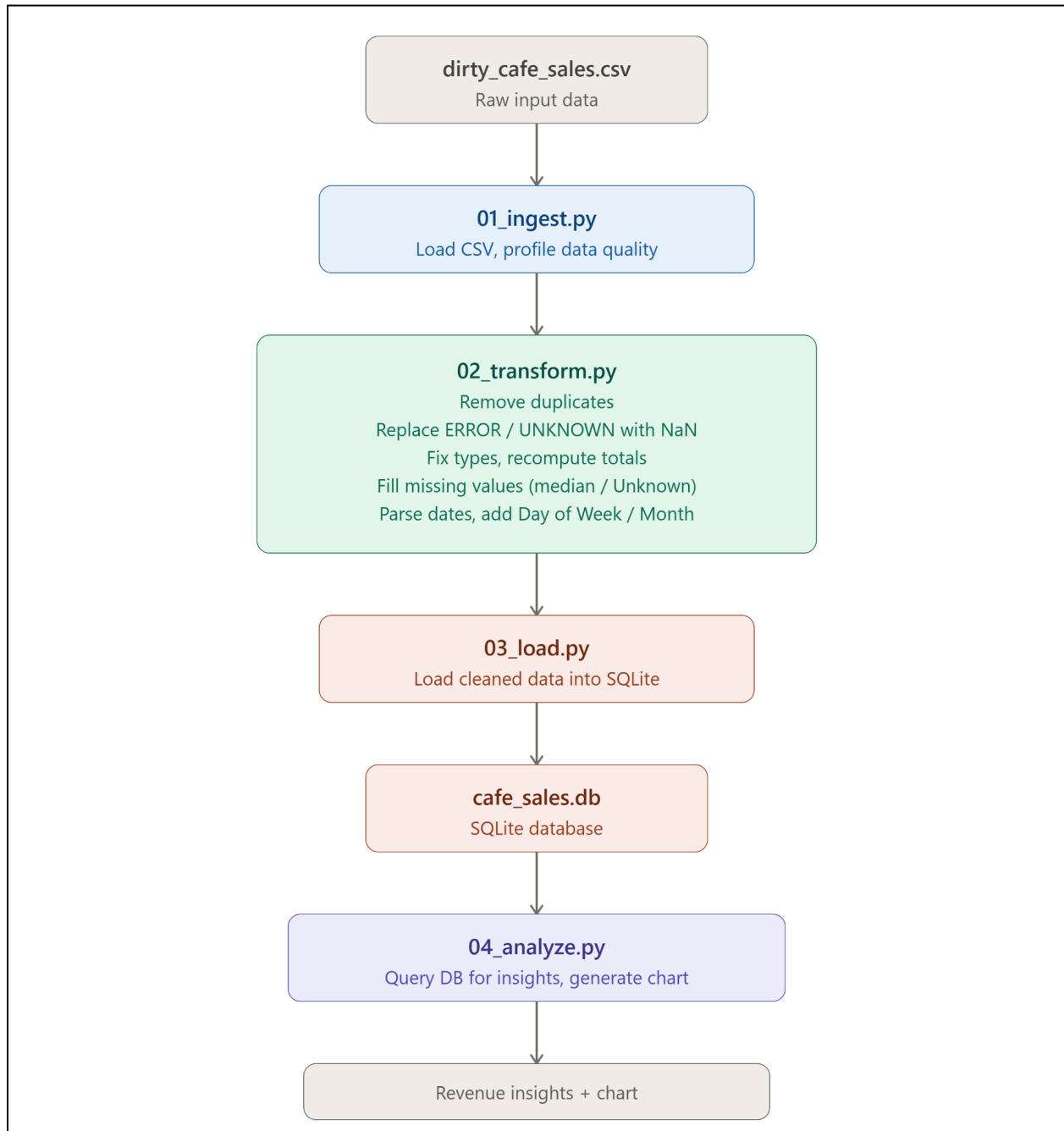


Figure 2 : Pipeline structure

5.1 Outputs

Table 2 presents the final outputs generated by the data processing pipeline.

File	Description
cleaned_cafe_sales.csv	Fully cleaned dataset with engineered columns
cafe_sales.db	SQLite database with cafe_sales table (1,824 rows)
revenue_by_item.png	Bar chart: total revenue per menu item

Table 2: Outputs of the pipeline

6.0 How Claude AI Was Used in This Pipeline

This section documents the prompt-based workflow used to build the pipeline, for use in the written report (Steps to do prompt and coding).

1. Ingestion script prompt

Asked Claude to write a script as in *Figure 3* to load the raw CSV and profile it (check missing values, data types, unique values, duplicates) so the cleaning steps could be planned based on actual data issues.

```
import pandas as pd

RAW_FILE = "dirty_cafe_sales.csv"

Trae: Explain | Doc | ✕
def load_raw_data(path: str) -> pd.DataFrame:
    """Load the raw CSV file into a DataFrame."""
    df = pd.read_csv(path)
    return df

Trae: Explain | Doc | ✕
def profile_data(df: pd.DataFrame) -> None:
    """Print a quick profile of the raw dataset (shape, dtypes, missing values)."""
    print("=" * 50)
    print("RAW DATA PROFILE")
    print("=" * 50)
    print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")
    print("\nColumn dtypes:")
    print(df.dtypes)
    print("\nMissing values per column:")
    print(df.isna().sum())
    print("\nSample rows:")
    print(df.head(5))
    print("\nUnique values in 'Item':", df["Item"].unique())
    print("Unique values in 'Payment Method':", df["Payment Method"].unique())
    print("Unique values in 'Location':", df["Location"].unique())
    print("Duplicate rows:", df.duplicated().sum())

if __name__ == "__main__":
    raw_df = load_raw_data(RAW_FILE)
    profile_data(raw_df)
```

Figure 3 : Ingestion script prompt

This script (01_ingest.py) imports the unprocessed dirty_cafe_sales.csv into a pandas DataFrame and displays a data-quality report. It states the total count of rows and columns, the data type associated with each column, the number of missing values for each column, a selection of rows, the distinct values in important categorical columns (Item, Payment Method, Location), and the count of duplicate rows. This profiling phase allows the team to identify the real data problems prior to determining how to resolve them.

2. Transformation prompt

Asked Claude to write a cleaning pipeline as in Figure 4, Figure 5 and Figure 6 that:

- removes duplicate rows
- standardizes invalid placeholders ("ERROR", "UNKNOWN") to NaN
- converts numeric columns properly
- recalculates missing totals from quantity x price
- fills remaining missing values (median for numeric, "Unknown" for categorical)
- parses and validates transaction dates
- engineers new columns (Day of Week, Transaction Month)

```
import pandas as pd
import numpy as np

from importlib import import_module

ingest = import_module("01_ingest")

Trae: Explain | Doc | X
def standardize_placeholders(df: pd.DataFrame) -> pd.DataFrame:
    """Replace 'ERROR' and 'UNKNOWN' text placeholders with proper NaN."""
    df = df.replace({"ERROR": np.nan, "UNKNOWN": np.nan})
    return df

Trae: Explain | Doc | X
def fix_numeric_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Convert Quantity, Price Per Unit, Total Spent to numeric, coercing bad values to NaN."""
    for col in ["Quantity", "Price Per Unit", "Total Spent"]:
        df[col] = pd.to_numeric(df[col], errors="coerce")
    return df

Trae: Explain | Doc | X
def recalculate_total_spent(df: pd.DataFrame) -> pd.DataFrame:
    """Where Total Spent is missing but Quantity and Price Per Unit are known, recompute it."""
    mask = df["Total Spent"].isna() & df["Quantity"].notna() & df["Price Per Unit"].notna()
    df.loc[mask, "Total Spent"] = df.loc[mask, "Quantity"] * df.loc[mask, "Price Per Unit"]
    return df
```

Figure 4 : Transformation prompt

Displays the initial three cleaning methods from 02_transform.py. `standardize_placeholders()` substitutes the erroneous "ERROR" and "UNKNOWN" strings with appropriate NaN values. `fix_numeric_columns()` changes Quantity, Price Per Unit, and Total Spent into numeric formats, converting any non-numeric garbage into NaN. `recalculate_total_spent()` reconstructs absent Total Spent values by multiplying Quantity $\times$ Price Per Unit wherever both are present.

```
def fill_missing_values(df: pd.DataFrame) -> pd.DataFrame:
    """Fill remaining missing numeric values with the column median,
    and missing categorical values with 'Unknown'."""
    numeric_cols = ["Quantity", "Price Per Unit", "Total Spent"]
    for col in numeric_cols:
        median_val = df[col].median()
        df[col] = df[col].fillna(median_val)

    categorical_cols = ["Item", "Payment Method", "Location"]
    for col in categorical_cols:
        df[col] = df[col].fillna("Unknown")

    return df

Trae: Explain | Doc | X
def clean_dates(df: pd.DataFrame) -> pd.DataFrame:
    """Parse Transaction Date into datetime; drop rows where the date is unusable."""
    df["Transaction Date"] = pd.to_datetime(df["Transaction Date"], errors="coerce")
    df = df.dropna(subset=["Transaction Date"])
    return df

Trae: Explain | Doc | X
def add_derived_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Add Day of Week and Transaction Month columns derived from Transaction Date."""
    df["Day of Week"] = df["Transaction Date"].dt.day_name()
    df["Transaction Month"] = df["Transaction Date"].dt.month_name()
    return df
```

Figure 5 : Transformation prompt

The cleaning process persists. `fill_missing_values()` addresses any leftover gaps by utilizing the median for numeric columns and "Unknown" for categorical columns. `clean_dates()` converts the Transaction Date into a proper datetime format and removes rows with irretrievable dates. `add_derived_columns()` creates two additional features, Day of Week and Transaction Month, based on the parsed date.

```

def remove_duplicates(df: pd.DataFrame) -> pd.DataFrame:
    """Remove exact duplicate rows, keeping the first occurrence."""
    before = len(df)
    df = df.drop_duplicates()
    after = len(df)
    print(f"Removed {before - after} duplicate rows.")
    return df

def transform(df: pd.DataFrame) -> pd.DataFrame:
    """Run the full transformation pipeline and return the cleaned DataFrame."""
    df = remove_duplicates(df)
    df = standardize_placeholders(df)
    df = fix_numeric_columns(df)
    df = recalculate_total_spent(df)
    df = fill_missing_values(df)
    df = clean_dates(df)
    df = add_derived_columns(df)
    df = df.reset_index(drop=True)
    return df

if __name__ == "__main__":
    raw_df = ingest.load_raw_data(ingest.RAW_FILE)
    print(f"Raw rows: {len(raw_df)}")

    clean_df = transform(raw_df)

    print(f"Clean rows: {len(clean_df)}")
    print("\nMissing values after cleaning:")
    print(clean_df.isna().sum())
    print("\nSample cleaned rows:")
    print(clean_df.head(10))

    clean_df.to_csv("cleaned_cafe_sales.csv", index=False)
    print("\nSaved cleaned data to cleaned_cafe_sales.csv")

```

Figure 6 : Transformation prompt

Demonstrates `remove_duplicates()`, which eliminates exact duplicate rows and indicates the count of removals, along with the master `transform()` function, which executes all cleaning procedures in the appropriate sequence to produce a completely cleaned DataFrame. The `__main__` section executes the entire transformation, displays the row counts before and after, as well as the remaining missing values, and finally saves the output to `cleaned_cafe_sales.csv`.

3. Loading prompt

Asked Claude to write a script as in *Figure 7* that loads the cleaned DataFrame into a SQLite database table for downstream querying.

```
import sqlite3
import pandas as pd

from importlib import import_module

ingest = import_module("01_ingest")
transform_module = import_module("02_transform")

DB_FILE = "cafe_sales.db"
TABLE_NAME = "cafe_sales"

Trae: Explain | Doc | X
def load_to_sqlite(df: pd.DataFrame, db_file: str = DB_FILE, table_name: str = TABLE_NAME) -> None:
    """Write the cleaned DataFrame to a SQLite database table (replacing any existing table)."""
    conn = sqlite3.connect(db_file)
    df.to_sql(table_name, conn, if_exists="replace", index=False)
    conn.close()
    print(f"Loaded {len(df)} rows into '{table_name}' table in {db_file}")

if __name__ == "__main__":
    raw_df = ingest.load_raw_data(ingest.RAW_FILE)
    clean_df = transform_module.transform(raw_df)
    load_to_sqlite(clean_df)
```

Figure 7 : Loading prompt

Displays 03_load.py, which accepts the processed DataFrame and inserts it into a SQLite database (cafe_sales.db) within the cafe_sales table via pandas' to_sql(), overwriting any existing table. This allows the cleaned data to be queried with SQL during the analysis phase.

4. Analysis/results prompt

Asked Claude to write SQL queries as in *Figure 8*, *Figure 9* and *Figure 10* against the SQLite database to calculate total revenue, revenue by item, revenue by day of week, by payment method, and by location, and to generate a chart of revenue by item.

```
import sqlite3
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from importlib import import_module

load_module = import_module("03_load")

DB_FILE = load_module.DB_FILE
TABLE_NAME = load_module.TABLE_NAME

Trae: Explain | Doc | X
def run_query(conn, query: str) -> pd.DataFrame:
    return pd.read_sql_query(query, conn)
```

Figure 8 : Analysis/results prompt

Displays the configuration of 04_analyze.py: importing necessary libraries, setting up matplotlib for non-interactive (file-based) chart generation, establishing a connection to the database, and defining a helper function run_query() that runs any SQL query and returns the results in a DataFrame.

```

if __name__ == "__main__":
    conn = sqlite3.connect(DB_FILE)

    total_revenue = run_query(conn, f"SELECT ROUND(SUM([Total Spent]), 2) AS total_revenue FROM {TABLE_NAME}")
    print("=" * 50)
    print("TOTAL REVENUE")
    print("=" * 50)
    print(total_revenue)

    revenue_by_item = run_query(conn, f"""
        SELECT Item, ROUND(SUM([Total Spent]), 2) AS revenue, SUM(Quantity) AS units_sold
        FROM {TABLE_NAME}
        GROUP BY Item
        ORDER BY revenue DESC
    """)
    print("\n" + "=" * 50)
    print("REVENUE BY ITEM")
    print("=" * 50)
    print(revenue_by_item)

    revenue_by_day = run_query(conn, f"""
        SELECT [Day of Week], ROUND(SUM([Total Spent]), 2) AS revenue
        FROM {TABLE_NAME}
        GROUP BY [Day of Week]
        ORDER BY revenue DESC
    """)
    print("\n" + "=" * 50)
    print("REVENUE BY DAY OF WEEK")
    print("=" * 50)
    print(revenue_by_day)

```

Figure 9 : Analysis/results prompt

Includes the main SQL queries: overall revenue from all transactions, revenue and units sold categorized by item, and revenue organized by day of the week, all ranked from highest to lowest. Outcomes are displayed in the console as labeled tables.

```

revenue_by_payment = run_query(conn, f"""
    SELECT [Payment Method], ROUND(SUM([Total Spent]), 2) AS revenue, COUNT(*) AS num_transactions
    FROM {TABLE_NAME}
    GROUP BY [Payment Method]
    ORDER BY revenue DESC
""")
print("\n" + "=" * 50)
print("REVENUE BY PAYMENT METHOD")
print("=" * 50)
print(revenue_by_payment)

revenue_by_location = run_query(conn, f"""
    SELECT Location, ROUND(SUM([Total Spent]), 2) AS revenue, COUNT(*) AS num_transactions
    FROM {TABLE_NAME}
    GROUP BY Location
    ORDER BY revenue DESC
""")
print("\n" + "=" * 50)
print("REVENUE BY LOCATION")
print("=" * 50)
print(revenue_by_location)

conn.close()

# Bar chart: revenue by item
plt.figure(figsize=(8, 5))
plt.bar(revenue_by_item["Item"], revenue_by_item["revenue"], color="#6f4e37")
plt.title("Total Revenue by Item")
plt.xlabel("Item")
plt.ylabel("Revenue")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig("revenue_by_item.png", dpi=150)
print("\nSaved chart to revenue_by_item.png")

```

Figure 10 : Analysis/results prompt

Displays the outstanding queries, revenue categorized by payment method, and revenue segmented by location (each accompanied by transaction totals), succeeded by the matplotlib code that creates the revenue-by-item bar chart and saves it as revenue_by_item.png.

5. Orchestration prompt

Asked Claude to combine all stages into a single `run\_pipeline.py` script as in Figure 11 that runs the full pipeline end-to-end and reports success or failure of each stage.

```
import subprocess
import sys

STAGES = [
    "01_ingest.py",
    "02_transform.py",
    "03_load.py",
    "04_analyze.py",
]

def main():
    for stage in STAGES:
        print("\n" + "#" * 60)
        print(f"# RUNNING STAGE: {stage}")
        print("#" * 60)
        result = subprocess.run([sys.executable, stage], capture_output=False)
        if result.returncode != 0:
            print(f"Stage {stage} failed. Stopping pipeline.")
            sys.exit(1)

    print("\nPipeline completed successfully.")
    print("Outputs: cleaned_cafe_sales.csv, cafe_sales.db, revenue_by_item.png")

if __name__ == "__main__":
    main()
```

Figure 11: Orchestration prompt

Displays `run_pipeline.py`, which executes the four scripts in order (ingest → transform → load → analyze) as subprocesses. Should any stage encounter an error, the pipeline halts and indicates which stage failed; if all stages succeed, it displays a success message and enumerates the resulting outputs.

7.0 Results and Analysis

7.1 Data Cleaning Summary

The *Table 3* below shows the result and summary of the dataset after cleaning process

Metric	Before Cleaning	After Cleaning
Total rows	2,020	1,824
Duplicate rows	20	0
Missing / invalid values	200–400 per column	0
Columns	8	10 (+ Day of Week, Month)

Table 3:Data Cleaning Summary Table

7.2 Revenue Analysis

After loading the 1,824 clean rows into SQLite, the analysis queries produced the following business insights. Total revenue across all transactions: RM 15,777.50.

1. Revenue by item :

Table 4 ranks each menu item by total revenue and units sold. The 'Unknown' category appears at the top because transactions with unrecoverable item names were imputed as 'Unknown' during cleaning and this is a data quality artifact rather than a genuine product. Among identifiable items, Smoothie leads in revenue while Cookie drives the highest unit volume relative to its revenue, reflecting its lower price point.

Table 4: Total Revenue by Day of Week (Ranked by Revenue)

TOTAL REVENUE			
total_revenue			
0		15777.5	
REVENUE BY ITEM			
	Item	revenue	units_sold
0	Unknown	4364.0	1492.0
1	Smoothie	2214.5	563.0
2	Salad	1968.0	465.0
3	Sandwich	1848.0	504.0
4	Cake	1397.0	483.0
5	Juice	1376.0	478.0
6	Coffee	1002.0	464.0
7	Tea	893.0	481.0
8	Cookie	715.0	505.0

Key insight: The 'Unknown' item category contributes the highest revenue (RM 4,364.00) due to a large number of transactions where the item could not be recovered and this is a data quality artifact from missing item names being imputed. Among identifiable items, Smoothie (RM 2,214.50) is the top performer, followed by Salad and Sandwich. Cookie generates the lowest revenue per transaction despite high unit volume (505 units), reflecting its lower price point.

2. Revenue by item -Bar chart

Figure 12 visualises the revenue distribution from Table 4 as a bar chart, generated by the analysis script using matplotlib. The chart confirms the dominance of the 'Unknown' category and shows a clear revenue gap between Smoothie and the remaining identifiable items

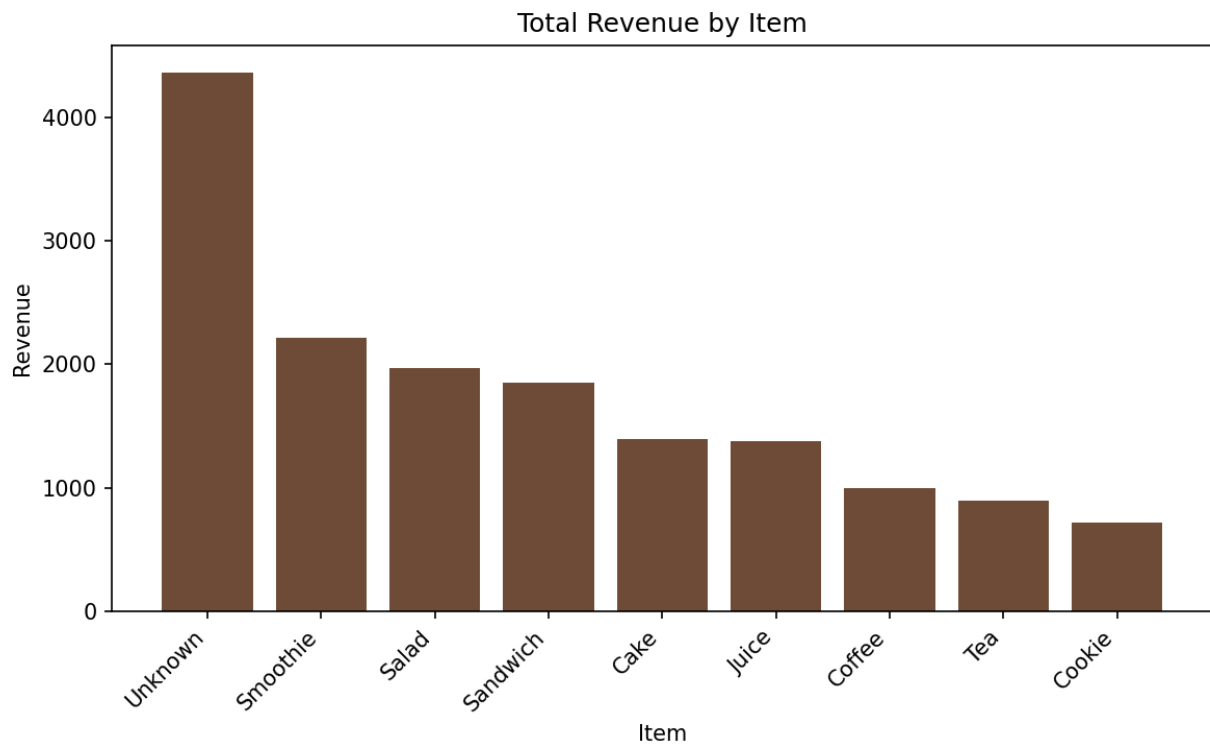


Figure 12: Total revenue by item generated by 04_analyze.py using matplotlib

3. Revenue by Day of Week :

Table 5 breaks down total revenue by the day on which each transaction occurred. Revenue is derived from the 'Day of Week' column that was engineered during the transformation stage. The distribution is notably even across all seven days, suggesting consistent trading patterns throughout the week with no strong peak day.

Table 5: Total Revenue by Day of Week (Ranked by Revenue)

REVENUE BY DAY OF WEEK		
	Day of Week	revenue
0	Monday	2387.5
1	Saturday	2336.0
2	Thursday	2326.0
3	Friday	2293.5
4	Tuesday	2272.5
5	Sunday	2091.0
6	Wednesday	2071.0

4. Revenue by Payment Method and Location :

Tables 6 and 7 present revenue breakdowns by payment method and transaction location respectively. Both dimensions are heavily affected by the original dataset's data quality issues with a large proportion of records had unrecoverable values and were labelled 'Unknown' during cleaning. Despite this, among recoverable records, Digital Wallet is the leading payment method, and Takeaway slightly outperforms In-store.

Table 6: Revenue and Transaction Count by Payment Method

=====			
REVENUE BY PAYMENT METHOD			
=====			
	Payment Method	revenue	num_transactions
0	Unknown	8048.5	931
1	Digital Wallet	2904.5	313
2	Credit Card	2562.0	307
3	Cash	2262.5	273

Table 7: Revenue and Transaction Count by Transaction Location

=====			
REVENUE BY LOCATION			
=====			
	Location	revenue	num_transactions
0	Unknown	9847.0	1133
1	Takeaway	3137.5	355
2	In-store	2793.0	336

Reflection

1. MUHAMMAD NAIM BIN ABDULLAH

Working on this pipeline showed how Claude can make data engineering much easier and faster. Instead of writing all the code by hand, Claude helped build each step of the pipeline, from reading the raw data, to cleaning it, to saving it in a database, and finally producing useful results. This is similar to what the tutorial showed, where AI can understand pipelines and help fix or build them just by chatting. Even though Claude did most of the coding, the results still needed to be checked to make sure the cleaning steps made sense, such as filling in missing values correctly. Overall, this shows that AI can save a lot of time, but human guidance and checking are still important.

2. MUHAMMAD ADAM BIN RAZALI

This project offered significant understanding of how Generative AI can function as an effective instrument within a well-organized data engineering process. My role focused on the prompt engineering component of the pipeline, particularly in creating clear and concise natural language directives to enable Claude to produce code that met the needs of each ETL phase. During this process, learned that the quality of the output generated relies significantly on the clarity of the problem statement; ambiguous prompts led to incomplete or faulty logic, while clearly defined prompts detailing the data quality issues, the expected handling approach, and the required output format resulted in much more dependable outcomes. I also acknowledged the significance of validating the generated code with the real dataset, since the AI cannot autonomously determine if a cleaning decision is suitable for the business context. In summary, this tutorial illustrated that although Claude notably lessens development effort and speeds up the creation of a functional pipeline, the engineer's responsibility in guiding, interpreting, and verifying its output is crucial for guaranteeing the accuracy and dependability of the end result.

3. AFIF SHAQIR IRFAN BIN ARQAM

Building this pipeline with Claude gave me a hands-on appreciation of how AI can genuinely change the speed of development in data engineering. My responsibility was to understand how each generated script connected to the next and to ensure the full pipeline ran end-to-end without errors in VS Code. One key takeaway is that AI does not eliminate the need for data engineering knowledge but it amplifies it. When Claude generated the transformation script, I still needed to understand why median imputation was chosen over mean, why rows with unparseable dates should be dropped rather than filled, and what the 'Unknown' category in the results actually means for the business. This tutorial demonstrated that the future of data engineering will involve AI as a collaborator but the engineer's judgement, domain understanding, and ability to critically validate outputs remain needed.